

北航数据结构期中测试编程题解析

编程题1

【问题描述】

有一种基于环的 **选择排序** 方法，其（从小至大排序）主要原理如下：

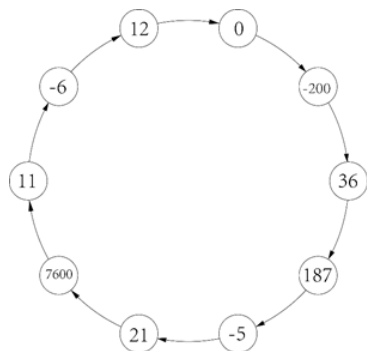
1. 首先将待排序的数据构成一个数据环；
2. 从环当前位置（初始时为待排序的第一个数据所在位置）开始按顺时针遍历环，从中找到当前环中最小元素；
3. 将当前位置移至最小元素的下一元素位置，并将最小元素从环中取出（得到一个排好序的元素）；
4. 重复步骤2和3，直到环中没有元素。

编写程序，从标准输入中读取n个无序且不同的整数，利用环选择排序方法对其进行从小至大排序。要求每得到一个当前最小元素就输出从当前元素开始，到最小元素所经过的环中数据。

假如读入下面10个整数：

12 0 -200 36 187 -5 21 7600 11 -6

可将这10个整数按照读入顺序顺时针连接起来形成如下图的环：



【输入形式】

先从控制台读入一个整数n(n大于等于1，小于等于100)，然后在下一行输入n个无序且不同的整数，各整数间以一个空格分隔。

【输出形式】

在屏幕上分行输出排序过程中每得到一个环中最小数据时所经历的环中的数据（包括该最小数据），各整数间以一个空格分隔，每行末尾整数后也有一个空格。

【样例输入】

10
12 0 -200 36 187 -5 21 7600 11 -6

【样例输出】

12 0 -200
36 187 -5 21 7600 11 -6
12 0 36 187 -5
21 7600 11 12 0
36 187 21 7600 11
12
36 187 21
7600 36
187
7600

【样例说明】

输入了10个整数，组成了如上图的环。从读入的第一个整数12开始，找到环中最小的整数-200，输出所经过的环中数据12 0 -200，然后将-200从环中删除，继续从下一个数据36开始，在环中找到最小的整数-6，并输出经过的环中数据，依次类推，直至环中没有数据，这样输出的每行最后一个元素形成了数据从小到大排序。

【评分标准】

该程序要求编程实现数据的循环查找删除操作，提交程序文件名为sort.c。

解析

【代码思路】

本题主要考察循环链表的创建，在循环链表中查找元素及删除元素等知识。代码设计思路如下：

1. 用尾插法构造循环链表；
2. 找到循环链表中的最小值的位置和最小值的前驱；
3. 打印从当前位置到最小值之间的结点；
4. 删除最小值结点；
5. 重复步骤2、3、4直到链表中只剩下一个结点（`curre->next == curre`）为止；
6. 打印最后一个结点，并释放存储空间。

【源代码】

```
1 //sort.c
2 #include<stdio.h>
3 #include<stdlib.h>
4 typedef struct ListNode{
5     int data;
6     struct ListNode *next;
7 }ListNode;
8
9 ListNode* CreateList()
10 {
11     int n;
12     scanf("%d", &n);
13     int data;
14     scanf("%d", &data);
15
16     ListNode*list, *p,*q;
17     list = (ListNode*)malloc(sizeof(ListNode));
18     list->next = NULL;
19     list->data = data;
20
21     q = list;
22     for (int i = 1; i < n; ++i)
23     {
24         p = (ListNode*)malloc(sizeof(ListNode));
25         scanf("%d", &data);
26         p->data = data;
27         p->next = NULL;
28         q->next = p;
29         q = p;
30     }
31     p->next = list;
32     return list;
33 }
34
35 ListNode* FindMin(ListNode*list, ListNode**pre)
36 { //找到最小的结点，*pre指向最小结点的前一个结点
37     ListNode*p = list;
38     ListNode* q,*ppre;
39     int min;
40     min = p->data;
41     ppre = NULL;
42     do
43     {
44         if (p->data <= min)
45         {
46             min = p->data;
47             q = p;
48             *pre = ppre;
49         }
50         ppre = p;
51         p = p->next;
52     } while (p != list);
53     if (*pre == NULL)
54     {
55         while (p->next != list) p = p->next;
56         *pre = p;
```

```

58 |     }
    |     59 |     return q;
60 | }
61 |
62 | void print(ListNode*curre, ListNode*minpos)
63 | { //打印从curre到minpos之间的结点（包含端点）
64 |     ListNode*p;
65 |     p = cure;
66 |     while (p != minpos)
67 |     {
68 |         printf("%d ", p->data);
69 |         p = p->next;
70 |     }
71 |     printf("%d \n", p->data);
72 | }
73 |
74 | void solve(ListNode*list)
75 | {
76 |     ListNode* minpos, *curre,*minpre;
77 |     curre = list;
78 |
79 |     while (curre->next != curre)
80 |     {
81 |         minpos = FindMin(curre, &minpre);
82 |         print(curre, minpos);
83 |
84 |         //删除minpos指向的结点
85 |         minpre->next = minpos->next; free(minpos);
86 |         curre = minpre->next;
87 |     }
88 |
89 |     //打印最后一个结点
90 |     printf("%d \n", curre->data); free(curre);
91 | }
92 |
93 | int main()
94 | {
95 |     freopen("1.txt", "r", stdin);
96 |     ListNode *list,*p;
97 |     list = CreateList();
98 |     solve(list);
99 | } //运行成功 2019年4月23日11:37:35

```

编程题2：格式控制输出模拟

【问题描述】

在C语言中，标准库函数printf可进行格式输出。编写程序，实现一种类似printf中的字符串格式输出，其从控制台读入一个格式控制字符和一行待输出的字符串，然后按照该格式控制串中的格式要求将该字符串输出到控制台。格式控制串的格式要求如下：

%[-]m:nS

格式控制串除了末尾有换行符外没有其他空白符；第一个字符是“%”；中括号表示其内字符可省略，字符“-”若省略，表示输出字符串靠左对齐，否则表示靠右对齐；m和n是大于0小于100的整数，两整数之间以字符“:”分隔；m表示只输出字符串中的前m个字符，若m大于字符串的长度，则表示字符串全部输出；n表示输出字符串至少占n个字符宽度，若n大于待输出的字符串长度，则多余的位置以字符“#”填充，若n小于等于待输出的字符串长度，则按照实际字符串输出，无需填充。

【输入形式】

从控制台读入格式控制串和待输出的字符串，第一行为格式控制串，第二行为待输出的字符串（字符串长度不超过100），第二行末尾的换行符不属于待输出的字符串。

【输出形式】

按照上述格式控制要求，将输入的第二行字符串输出到控制台。

【样例1输入】

%-20:30S

Hello,word!

【样例1输出】

#####Hello,word!

【样例1说明】

根据第一行的格式控制要求，要将第二行的前20个字符按照右对齐输出，最少占30个字符的宽度，因为待输出的字符串只有12个字符（注意：字符w前有个空格），所以要全部输出，并且在左边填充18个“#”字符。

【样例2输入】

```
%8:30S
```

```
Hello,word!
```

【样例2输出】

```
Hello,w#####
```

【样例2说明】

根据第一行的格式控制要求，要将第二行的前8个字符按照左对齐输出，最少占30个字符的宽度。待输出的字符串有12个字符，只输出前8个字符，并且在右边填充22个“#”字符。

【评分标准】

该程序要求编程实现字符串格式控制输出，提交程序文件名为outputf.c。

解析

【代码思路】

本题主要考察字符串解析及C语言基本库函数的熟练程度，思维比第一题简单，代码思路如下：

- 1.对格式串进行解析，得到对齐方式、m和n的数值等信息。
- 2.用待输出字符串的长度与m、n进行数值比较，判断是否要填充，是否要对字符串进行截断，最后再根据对齐方式等规则按要求打印出待输出字符串即可。

【源代码】

```
1  #include<stdio.h>
2  #include<string.h>
3  char buff[1000];
4  char commadline[100];
5  void print(int m, int len)
6  {
7      if (m > len)
8          printf("%s", buff);
9      else
10     {
11         //输出前m个字符
12         int i;
13         for ( i = 0; i < m; ++i)
14             putchar(buff[i]);
15     }
16 }
17
18 void tianchong(int n)
19 {
20     int i;
21     for ( i = 0; i < n; ++i)
22         putchar('#');
23 }
24
25 int main()
26 {
27     freopen("2.txt", "r", stdin);
28     gets(commadline);
29     gets(buff);
30
31     bool right = false;
32     if (commadline[1] == '-') right = true;
33     char newcomm[100];
34     if (right)
35         strcpy(newcomm, commadline+2);
36     else
37         strcpy(newcomm, commadline + 1);
38     int m, n;
39     sscanf(newcomm, "%d:%d", &m, &n);
40
41     int len = strlen(buff);
42
43     int lenc; //填充的字符个数
44     if (m > len)
45         lenc = n - len;
46     else
47         lenc = n - m;
```

```
48 |
49 |     if (n > len)
50 |     {
51 |         //n大于字符长度，要填充
52 |         if (!right)
53 |         { //先输出再填充
54 |             print(m, len);
55 |             tianchong(lenc);
56 |         }
57 |         else
58 |         {
59 |             tianchong(lenc);
60 |             print(m, len);
61 |         }
62 |     }
63 | else
64 | {
65 |     print(m, len);
66 | }
67 | }
```

2018.04.22

文章知识点与官方知识档案匹配，可进一步学习相关知识

算法技能树 首页 概览 43256 人正在系统学习中

“相关推荐”对你有帮助么？



非常没帮助



没帮助



一般



有帮助



非常有帮助